

MEDIATEK

MT8676 Yocto T-Box User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Liuyutian Liu	Official release

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

List of Tables 3

1 T-Box..... 4

1.1 Overview 4

1.1.1 Brief Introduction 4

1.1.2 T-Box Abbreviations 4

1.2 Architecture/Process Overview..... 5

1.2.1 T-Box Architecture..... 5

1.2.2 T-Box API Usage..... 5

1.3 Frequently asked question/Troubleshoot 16

1.3.1 SIM/CALL/SMS/Telephony Network Tips 16

1.3.2 Data Tips 16

1.3.3 Network Tips 16

1.3.4 IMS Tips..... 17

Exhibit 1 Terms and Conditions 18

List of Figures

Figure 1-1. Yocto T-Box architecture 5

List of Tables

Table 1-1. Abbreviations..... 4

Table 1-2. Modem state and IMEI interface description 5

Table 1-3. SIM interface description 6

Table 1-4. Telephony NW interface description 7

Table 1-5. Data interface description 8

Table 1-6. Call module interface description..... 10

Table 1-7. Call interface description 11

Table 1-8. IMS interface description 12

Table 1-9. eCall control interface description..... 12

Table 1-10. AT blacklist interface description 15

Table 1-11. MIPC Keep alive interface description 15

1 T-Box

1.1 Overview

1.1.1 Brief Introduction

This section introduces the Yocto T-Box architecture, API usage and precautions.

1.1.2 T-Box Abbreviations

Table 1-1. Abbreviations

Abbreviation	Explanation
API	Application Programming Interface
APN	Access Point Name
T-Box	Telematics-BOX

1.2 Architecture/Process Overview

1.2.1 T-Box Architecture

Hypervisor Dual System Tbox Architecture

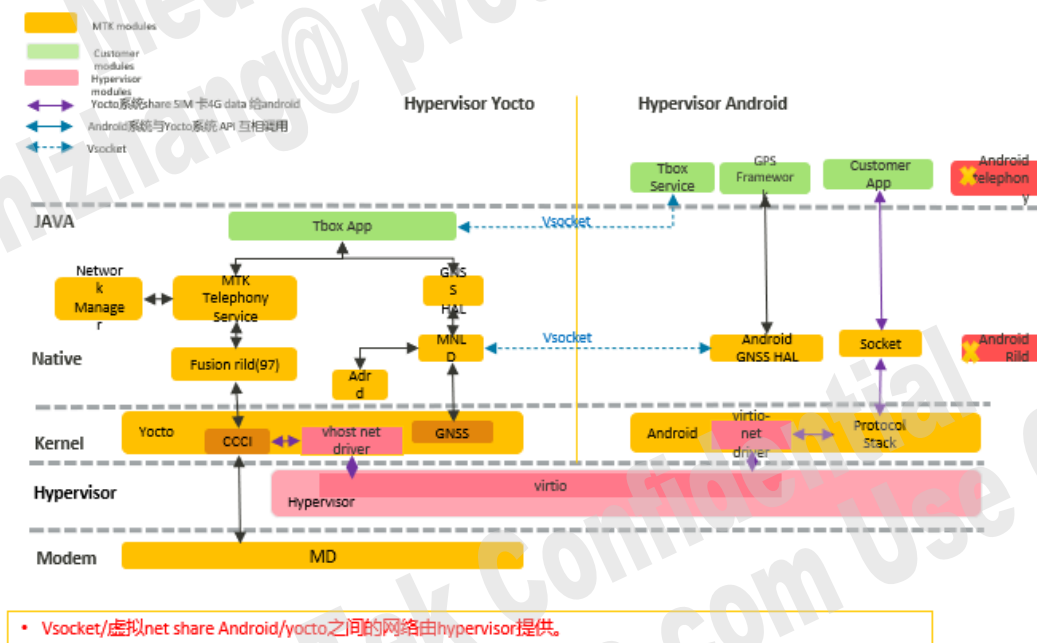


Figure 1-1. Yocto T-Box architecture

1.2.2 T-Box API Usage

1.2.2.1 Modem State/IMEI Interface and Call Sequence Description

All other T-Box APIs require modem ready to work properly except ML_GetModemStat.

Table 1-2. Modem state and IMEI interface description

Interface/Struct	Description
Int ML_GetModemStat(char *stat, uint32_t statlen)	Get modem state, stat will return "ready" if modem has boot up success.
Int ML_GetImei(char* imei, size_t imeiLen)	Get IMEI of device. On success, 0 is returned. On error, -1 is returned.

1.2.2.2 SIM Interface and Call Sequence Description

All other SIM APIs require SIM present to work properly except ML_GetCardStatus.

Table 1-3. SIM interface description

Interface/Struct	Description
<pre>typedef enum { E_ML_SIM_CARD_STATE_UNKNOWN = 0xB01, /**< Card state unknown. */ E_ML_SIM_CARD_STATE_ABSENT = 0xB02, /**< Card is absent. */ E_ML_SIM_CARD_STATE_PRESENT = 0xB03, /**< Card is present. */ E_ML_SIM_CARD_STATE_ERROR_UNKNOWN = 0xB04, /**< Unknown error state. */ E_ML_SIM_CARD_STATE_ERROR_POWER_DOWN = 0xB05, /**< Power down. */ E_ML_SIM_CARD_STATE_ERROR_POLL_ERROR = 0xB06, /**< Poll error. */ E_ML_SIM_CARD_STATE_ERROR_NO_ATR_RECEIVED = 0xB07, /**< Failed to receive an answer to reset. */ E_ML_SIM_CARD_STATE_ERROR_VOLT_MISMATCH = 0xB08, /**< Voltage mismatch. */ E_ML_SIM_CARD_STATE_ERROR_PARITY_ERROR = 0xB09, /**< Parity error. */ E_ML_SIM_CARD_STATE_ERROR_SIM_TECHNICAL_PROBLEMS = 0xB0A, /**< Card returned technical problems. */ }E_ML_SIM_CARD_STATE_TYPE_T; /**< Card state. */</pre>	SIM card state struct.
Int32_t ML_Sim_GetCardStatus(ML_SIM_CARD_STATUS_INFO_T *pvsSimStatus)	Get SIM state. The value of pvsSimStatus will return "present" when SIM card is present. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetICCID(char *iccid, size_t iccidLen)	Get ICCID of SIM card. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetMsisdn(char *msisdn, size_t msisdnLen)	Get MSISDN of SIM card. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetImsi(char *imsi, size_t imsilen)	Get IMSI of SIM card. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetMccMnc(char *mcc, size_t mccLen, char *mnc, size_t mncLen)	Get MCCMNC of SIM card. Function return value is int type. On success, 0 is returned. On error, -1 is returned.

1.2.2.3 Telephony NW Interface and Call Sequence Description

Table 1-4. Telephony NW interface description

Interface/Struct	Description
<pre>typedef enum { E_ML_NW_PS_UNKNOWN = 0x00, E_ML_NW_PS_ATTACHED = 0x01, E_ML_NW_PS_DETACHED = 0x02, }E_ML_NW_PS_REG_STATE_TYPE_T;</pre>	Struct of PS registration state.
<pre>typedef enum { E_ML_NW_CS_UNKNOWN = 0x00, E_ML_NW_CS_ATTACHED = 0x01, E_ML_NW_CS_DETACHED = 0x02, }E_ML_NW_CS_REG_STATE_TYPE_T;</pre>	Struct of CS registration state.
<pre>int ML_GetNetState(E_ML_NW_PS_REG_STATE_TYPE_T * ps, E_ML_NW_CS_REG_STATE_TYPE_T * cs)</pre>	Get CS/PS registration state. Function return value is int type. On success, 0 is returned. On error, -1 is returned. Notice: This interface should be used after SIM present.
<pre>int32_t ML_GetOperatorCode (uint8_t * op_code)</pre>	Get operator name. Value of op_code is "1" when sim operator is CU. Value of op_code is "0" when SIM operator is CMCC. Function return value is int type. On success, 0 is returned. On error, -1 is returned. Notice: This interface should be used after SIM present.
<pre>typedef enum { E_ML_NW_RAT_GSM = 0, E_ML_NW_RAT_UTRAN = 2, E_ML_NW_RAT_GSMW_EGPRS = 3, E_ML_NW_RAT_UTRANW_HSDPA = 4, E_ML_NW_RAT_UTRANW_HSUPA = 5, E_ML_NW_RAT_UTRANW_HSDPA_AND_HSUPA = 6, E_ML_NW_RAT_E_UTRAN = 7, #ifdef SPM_TELEPHONY_NR_SUPPORT E_ML_NW_RAT_NR = 8, #endif }E_ML_NW_RADIO_ACCESS_TYPE_T;</pre>	Struct of radio access type.
<pre>int ML_GetRadioAccessType (E_ML_NW_RADIO_ACCESS_TYPE_T * access_type)</pre>	Get radio access type of network registered. Function return value is int type. On success, 0 is returned. On error, -1 is returned. Notice: This interface should be used after SIM present.

Interface/Struct	Description
int ML_GetSignalStrength (int8_t * sig_level)	Get signal strength level. Sig_level: 0~4. For signal level thresholds, refer to AOSP default classification. Function return value is int type. On success, 0 is returned. On error, -1 is returned. Notice: This interface should be used after SIM present.
int ML_SetAirplaneMode (uint8_t on_off)	Set airplane mode. Input value "on_off" will be set to 1 if airplane on. Input value "on_off" will be set to 0 if airplane off. Function return value is int type. On success, 0 is returned. On error, -1 is returned. Notice: This interface should be used after SIM present.
typedef void (*ml_signal_strength_cb_t)(ML_SignalStrength *state);	Signal strength call back function.
int ML_SignalStreghlInit (ml_signal_strength_cb_t evt_cb)	Init signal strength call back function. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
typedef enum { E_ML_AUTO = 0, E_ML_2GONLY = 1, E_ML_3GONLY = 2, E_ML_4GONLY = 3, #ifdef SPM_TELEPHONY_NR_SUPPORT E_ML_5GONLY = 4, E_ML_5GAUTO = 5, E_ML_5G4GAUTO = 6, #endif }ml_nw_net_mode_e;	Struct of network mode.
int ML_SetNetMode (ml_nw_net_mode_e mode)	Set network mode. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int ML_GetNetMode (ml_nw_net_mode_e * mode)	Get network mode. Function return value is int type. On success, 0 is returned. On error, -1 is returned.

1.2.2.4 Data Interface and Call Sequence Description

Table 1-5. Data interface description

Interface/Struct	Description
typedef struct { unsigned char profile_idx; ml_apn_pdp_type_e pdp_type;	Struct of APN info profile_idx: 1 – 8 (1: Public profile ID; 2-8: Private profile ID, support up to 7 private APNs)

Interface/Struct	Description
<pre>ml_apn_auth_proto_e auth_proto; char apn_name[ML_APN_NAME_SIZE]; char username[ML_APN_USERNAME_SIZE]; char password[ML_APN_PASSWORD_SIZE]; }ml_apn_info_s</pre>	pdp_type: IPV4/IPV6/IPV4V6; for details, refer to ml_apn_pdp_type_e enum auth_proto: default/none/pap/chap/pap_chap apn_name: APN name
<pre>int ML_APN_Get (unsigned char profile_idx, ml_apn_info_s * apn)</pre>	Get APN info from input value "profile_idx". Parameters: - in: unsigned char profile_idx - out: ml_apn_info_s * apn APN initial flow: After the card is recognized at startup, mtktelephony service will determine whether /data/vendor/telephony/apn.db exists. If not, it generates /data/vendor/telephony/apn.db according to /vendor/etc/apns_conf.xml. Then it obtains the APN of the SIM card from apn.db through the card's mccmnc and sets the APN to the modem. Notice: This interface needs to be called after the card is recognized, that is, after ML_Sim_GetCardStatus() returns present.
<pre>int ML_APN_Set (const ml_apn_info_s * apn)</pre>	Set APN to modem and apn database(/data/vendor/telephony/apn.db) Parameters: in: const ml_apn_info_s * apn Notice: This interface needs to be called after the card is recognized
<pre>int ML_DataCallInit (ml_data_call_evt_cb_t evt_cb)</pre>	Register the data call status change callback function, and call back the evt_cb function when the data call status changes. Parameters: in: ml_data_call_evt_cb_t evt_cb
<pre>int ML_DataCallStart (const ml_data_call_s * data_call, ml_data_call_error_e * err)</pre>	Establish public/private PDN connection. This is synchronous interface. It sends PDN establishment command to modem, and this interface returns after modem returns. Parameters: - in: const ml_data_call_s * data_call - out: ml_data_call_error_e * err
<pre>int ML_DataCallStop (char profile_idx, ml_data_call_ip_family_e ip_family, ml_data_call_error_e * err)</pre>	Disconnect the public/private PDN connection. This is synchronous interface. It sends the PDN establishment command to the modem, and this interface will return only after the modem returns. Parameters: - in: char profile_idx - in: ml_data_call_ip_family_e ip_family - out: ml_data_call_error_e * err

Interface/Struct	Description
int ML_DataCallStart_Ext (const ml_data_call_s * data_call, ml_data_call_error_e * err)	Establish public/private PDN connection. This asynchronous interface, returns directly after issuing the PDN establishment command, without waiting for the modem execution result. The status of establishing PDN is called back to the APP through the callback function registered by ML_DataCallInit. Parameters: in: const ml_data_call_s * data_call out: ml_data_call_error_e * err
int ML_DataCallStop_Ext (char profile_idx, ml_data_call_ip_family_e ip_family, ml_data_call_error_e * err)	Disconnect public/private PDN connection Asynchronous interface, returns directly after issuing the disconnect PDN command, without waiting for the modem execution result. The disconnect PDN status is called back to the APP through the callback function registered by ML_DataCallInit. Parameters: in: char profile_idx in: ml_data_call_ip_family_e ip_family out: ml_data_call_error_e * err
int ML_getDataCallReason (int32_t profile_idx, ML_DataCallFailCause * reason)	When PDN establishment fails, get failcause. Parameters: in: profile_idx in/out: reason
Int ML_DataCallInfoGet(char profile_idx, ml_data_call_ip_family_e ip_family, ml_data_call_info_s *info, ml_data_call_error_e *err)	Get data call information according to profile_idx and ip_family. Parameters: in: char profile_idx in: ml_data_call_ip_family_e ip_family out: ml_data_call_info_s *info out: ml_data_call_error_e *err

1.2.2.5 Call Interface and Call Sequence Description

All call module interfaces need to be called based on network registration.

Table 1-6. Call module interface description

Interface/Struct	Description
typedef struct { int32_t CallId; char PhoneNum[32]; E_ML_VCALL_STATE_TYPE_T State; int32_t call_end_reason; } ML_VCALL_INFO_T;	Struct of call message information. CallId: Current call index; PhoneNum[32]: The opponent's phone number; State: Call state; call_end_reason: The reason of call end;
typedef void (*ML_VCALL_MSGCB_T) (ML_VCALL_INFO_T *pvsMsg);	Call state callback function.

Interface/Struct	Description
Int32_t ML_VcallInit(ML_VCALL_MSGCB_T cb_func)	Register the call state callback function. The callback function is triggered when a call comes in, and the call information is returned through the callback function. Parameters: out: ML_VCALL_MSGCB_T cb_func;
Int ML_VcallStart(const char *PhoneNumber)	Dial a call. Parameters: in: const char *PhoneNumber;
Int ML_VcallAnswer(void)	Answer a call.
Int ML_VcallEnd (void)	Hang up a call.

1.2.2.6 SMS Interface and Call Sequence Description

The SMS interfaces must be called based on the network registration.

Table 1-7. Call interface description

Interface/Struct	Description
typedef enum { E_ML_SMS_FORMAT_GSM_7BIT = 0, E_ML_SMS_FORMAT_BINARY_DATA = 1, E_ML_SMS_FORMAT_UCS2 = 2, E_ML_SMS_FORMAT_IRA = 3, }E_ML_SMS_FORMAT_T;	Enum of SMS format type.
typedef struct { E_ML_SMS_FORMAT_T format; char PhoneNum[ML_SMS_MAX_ADDR_LENGTH]; int32_t SmsDataLen; char SmsData[ML_SMS_MAX_MT_MSG_LENGTH]; } ML_SMS_INFO_T;	Struct of SMS information.
typedef void (*ML_SMS_RXMSGCB_T) (ML_SMS_INFO_T *pvsMsg);	SMS callback function.
Int ML_SmsInit(ML_SMS_RXMSGCB_T cb_func)	Register SMS callback function. The callback function is triggered when receiving SMS, and the SMS information is returned through the callback function. Parameters: out: ML_SMS_RXMSGCB_T cb_func;
Int32_t ML_Sms_Sent(const ML_SMS_INFO_T *pvsSms)	Send SMS. This is an asynchronous interface. After sending SMS request, it will asynchronously wait for md to return response, and will not block the thread.

Interface/Struct	Description
	Parameters: in: const ML_SMS_INFO_T *pvsSms;

1.2.2.7 IMS Interface and Call Sequence Description

Table 1-8. IMS interface description

Interface/Struct	Description
Int ML_EnableIms(uint8_t on_off)	Enable/disable the IMS registration. Input parameter: On_off is 1, which means the IMS function is enabled. On_off is 0, which means the IMS function is disabled.
Int ML_GetImsRegState(int8_t *reg_state)	Get the IMS registration status. Output parameter: Reg_state value is 1, which means IMS is registered. Reg_state value is 0, which means IMS is not registered.

1.2.2.8 Ecall Interface and Call Sequence Description

The following APIs are planned to be added in MT8676, based on the ECALL-related RIL Request in the ecall overall sequence in the *Ecall Overview* document.

Table 1-9. eCall control interface description

Interface/Struct	Description
typedef struct { int32_t call_id; uint32_t length; unsigned char msd_data[ML_ECALL_MSD_MAX_LENGTH]; }ml_ecall_set_msd;	Struct of MSD information.
typedef struct { int32_t arg_num; int32_t type; char address[128]; }ml_ecall_set_num;	Struct of test number/reconfiguration number information.
typedef enum { ML_EMER_CAT_MANUAL_ECALL = 1, ML_EMER_CAT_AUTO_ECALL = 2, }ml_ecall_category;	Enum of ecall category.
typedef enum { ML_ECALL_TEST = 1, ML_ECALL_EMERGENCY = 2, ML_ECALL_RECONFIG = 3,	Enum of ecall variant.

Interface/Struct	Description
<pre> }ml_ecall_variant; typedef enum{ ML_DOMAIN_AUTO = 0, /* Automatic mode - LTE(IMS), WG(CS), 1x(C2K) */ ML_DOMAIN_CS_ONLY = 1, /* CS domain only - WG(CS) */ ML_DOMAIN_3GPP_ONLY = 2, /* 3GPP only - LTE(IMS), WG(CS) */ ML_DOMAIN_3GPP2 = 3, /* 3GPP2 only - 1x(C2K)) */ ML_DOMAIN_IMS_1xCS = 4, /* IMS and 1x CS only - LTE(IMS), 1x(C2K) */ ML_DOMAIN_CS_1x = 5, /* WG CS and 1x CS only - WG(CS), 1x(C2K) */ ML_DOMAIN_IMS_ONLY = 6, /* only IMS call allowed */ }ml_ecall_domain; </pre>	Enum of radio domain when make ecall.
<pre> typedef struct{ ml_ecall_category ecall_cat; ml_ecall_variant ecall_variant; char address[20]; uint32_t length; unsigned char msd_data[ML_ECALL_MSD_MAX_LENGTH]; ml_ecall_domain domain; }ml_ecall_req_msg; </pre>	Struct of input parameter when requesting to make an ecall.
<pre> typedef struct{ int32_t data1; int32_t data2; int32_t data3; int32_t data4; }ml_ecall_pri; </pre>	Ecall priority parameter structure. data1>data2>data3>data4. The input parameters should be 1 2 3 4, which means: 1 - eCall URI set by the customer; 2 - eCall URI saved by USIM; 3 - eCall number set by the customer; 4 - eCall number saved by USIM;
<pre> typedef enum{ E_ML_ECALL_SENDING_START = 1, E_ML_ECALL_SENDING_MSD = 2, E_ML_ECALL_LACK_RECEIVED = 3, E_ML_ECALL_ALACK_POSITIVE_RECEIVED = 4, E_ML_ECALL_ALACK_CLEARDOWN_RECEIVED = 5, E_ML_ECALL_DIALING = 9, E_ML_ECALL_ALERTING = 10, E_ML_ECALL_ACTIVE = 11, E_ML_ECALL_DISCONNECTED = 12, E_ML_ECALL_IMS_ACTIVE = 13, </pre>	Enum of ecall indication type.

Interface/Struct	Description
<pre> E_ML_ECALL_IMS_DISCONNECTED = 14, E_ML_ECALL_ABNORMAL_HANGUP=15, E_ML_ECALL_IMS_MSD_ACK = 20, E_ML_ECALL_IMS_UPDATE_MSD = 21, E_ML_ECALL_IMS_IN_BAND_TRANSFER = 22, E_ML_ECALL_IMS_MSD_NACK = 23, E_ML_ECALL_IMS_SRVCC = 24, E_ML_ECALL_ONLY_DEREGISTRATION = 31, E_ML_ECALL_MAY_DEREGISTER = 32, E_ML_ECALL_PSAP_CALLBACK_START = 40, E_ML_ECALL_PSAP_CALLBACK_IMS_UPDATE_MSD = 41, E_ML_ECALL_T2_TIMEOUT = 52, E_ML_ECALL_T5_TIMEOUT = 55, E_ML_ECALL_T6_TIMEOUT = 56, E_ML_ECALL_T7_TIMEOUT = 57, E_ML_ECALL_UNSPECIFIED = 0xffff, }ML_ECall_Indication; </pre>	
<pre> typedef struct{ ML_ECall_Indication ind; int call_id; } ML_ECALL_IND_T; </pre>	Struct of ecall indication.
<pre> typedef void (*ML_ECALL_MSGCB_T)(ML_ECALL_IND_T *pvsMsg); </pre>	Callback function for reporting ecall indication.
<pre> int32_t ML_EcallIndicationInit(ML_ECALL_MSGCB_T cb_func); </pre>	Register the ecall indication callback function, and call back the cb_func function when the ecall status changes. Parameters: In: ML_ECALL_MSGCB_T cb_func
<pre> int32_t ML_ResetIvs(void); </pre>	Reset ecall state in modem and reconnect audio channel.
<pre> int32_t ML_SetMSD(ml_ecall_set_msd* msd); </pre>	Set MSD. Parameters: In: ml_ecall_set_msd* msd
<pre> int32_t ML_SetTestNumber(ml_ecall_set_num* test_num); </pre>	Set test number or URI. Parameters: In: ml_ecall_set_num* test_num
<pre> int32_t ML_SetReconfNumber(ml_ecall_set_num* reconf_num); </pre>	Set reconfig number or URI. Parameters: In: ml_ecall_set_num* test_num
<pre> int32_t ML_MakeFastEcall(ml_ecall_req_msg* msg); </pre>	Trigger an ecall and transmit MSD data in one command. Parameters: In: ml_ecall_req_msg* msg

Interface/Struct	Description
int32_t ML_SetEmsdpri(ml_ecall_pri* pri);	Set priority of test/reconfiguration eCall number/URI The default priority is “1>3>2>4” . Parameters: In: ml_ecall_pri* pri

1.2.2.9 Suspend of Modem Interface and Call Sequence Description

Table 1-10. AT blacklist interface description

Interface/Struct	Description
Int ML_SendAT(const char* atCmd, char* finalRsp, uint32_t resp_len, int64_t timeout_ms); Int ML_SetUnsolicitedResponseFilter(ML_UnsolicitedResponseFilter filter)	Before entering IPO, enable the AT URC blacklist filtering function: 1. Call ML_SendAT function to send AT command: AT+EURCFLT=1. 2. Call ML_SetUnsolicitedResponseFilter(0) After exiting IPO, disable the AT URC blacklist filtering function: 1. Call ML_SendAT function to send AT command: AT+EURCFLT=0 2. Call ML_SetUnsolicitedResponseFilter(0xFF)

Table 1-11. MIPC Keep alive interface description

Interface/Struct	Description
ML interface: Plan to encapsulate ML layer API based on MIPC interface. struct keepaliveRequest { keepaliveType type; uint32_t sourcePort; uint32_t destinationPort; uint32_t cid; char sourceAddress[68]; char destinationAddress[68]; } struct keepaliveStatus{ ACTIVE = 0, INACTIVE, PENDING } ML_StartKeepalive(keepaliveRequest * keepalive) ML_StopKeepalive(int cid) ML_InitKeepaliveStatusCb(ml_keepalive_evt_cb_t evt_cb)	To keep Internet PDN/PDU session in connected mode by sending a dummy packet without waking up AP processor. Flow: 1. AP calls ML_StartKeepalive. The callback keepaliveStatus status is pending, indicating that it is waiting for the AP to send the first UL packet. 2. After the MD receives the first UL packet sent by the AP, the callback keepaliveStatus status is active, indicating that the MD can send keep-alive packets. 3. If an exception occurs, the callback keepaliveStatus status is inactive when the modem finally fails to receive the network keep-alive response.

Interface/Struct	Description
MIPC interface: MIPC_DATA_KEEPALIVE_REQ/CNF MIPC_DATA_KEEPALIVE_STATUS_IND	

1.3 Frequently asked question/Troubleshoot

1.3.1 SIM/CALL/SMS/Telephony Network Tips

- How to turn off/on a specific network type, refer to “[FAQ17447] [NW] Turn off a specific network type”.
- SIM test failure pre-debug:
 - The initial value of IMEI is empty and needs to be burned before it can be obtained.
 - If SIM recognition is abnormal, capture the test scene mtklog including the boot process to assist in further analysis.
- Network test failure pre-debug:
 - Confirm that the platform has burned RF/IMEI, the antenna is correctly installed, and the SIM card is correctly inserted and has no outstanding fees.
 - If (1) still fails to attach network after confirmation, capture the test scene mtklog including the boot process to assist in further analysis.
- CALL/SMS test failure pre-debug:
 - Use the following command to confirm whether the platform has successfully attached network. If not, solve the network problem first.

```
adb shell /usr/bin/mlclient_test --gtest_filter=MLClientTest.ML_GetNetState
```
 - If step (1) has been successfully attached to the network. Capture the test scene mtklog to assist in further analysis.

1.3.2 Data Tips

Modification of APN configuration file, refer to “[FAQ21414] MTK apns-conf.xml configuration guide”.

The configuration file used by T-Box is named apns-conf.xml, code path: `src/telephonyware/3.1/libvendor-ril/apn/resource/apns-conf.xml`, install path in the platform `/system/etc/tele/apns-conf.xml`.

If you modify `/system/etc/tele/apns-conf.xml` directly, you need to delete `apn.db` and restart to take effect.

1.3.3 Network Tips

- Checking for successful network establishment:
 - The network interface is up and the IP address exists, confirmed by the `ifconfig` command.
 - Network IP rule & IP route & DNS settings, confirmed by the `IP rule/IP route/dumpsys netd/dumpsys dnsresolver` commands.
- Checking for network connectivity:

Manually ping a certain IP or URL to see if it can be pinged.

1.3.4 IMS Tips

1. How to enable or disable IMS automatically when the device is powered on by default?
 - (1) The current system automatically enables IMS when the device is powered on by default.
 - (2) You can change whether IMS is supported by changing MTK_VOLTE_SUPPORT to yes or no in ProjectConfig.mk.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.